Multi-Artifact Analysis of Self-Admitted Technical Debt in Scientific Software

Eric L. Melin
Boise State University, Oak Ridge National Laboratory
Boise, ID, USA
ericmelin@u.boisestate.edu

Nasir U. Eisty
University of Tennessee
Knoxville, TN, USA
neisty@utk.edu

Gregory Watson
Oak Ridge National Laboratory
Oak Ridge, TN, USA
watsongr@ornl.gov

Addi Malviya-Thakur
Oak Ridge National Laboratory, University of Tennessee
Knoxville, TN, USA
malviyaa@ornl.gov

Abstract

Context: Self-admitted technical debt (SATD) occurs when developers acknowledge shortcuts in code. In scientific software (SSW), such debt poses unique risks to the validity and reproducibility of results. ***Objective:*** This study aims to identify, categorize, and evaluate *scientific debt*, a specialized form of SATD in SSW, and assess the extent to which traditional SATD categories capture these domain-specific issues. ***Method:*** We conduct a multi-artifact analysis across code comments, commit messages, pull requests, and issue trackers from 23 open-source SSW projects. We construct and validate a curated dataset of *scientific debt*, develop a multi-source SATD classifier to guide SATD management, and conduct a practitioner validation to assess the practical relevance of *scientific debt*. ***Results:*** Our classifier performs strongly across 900,358 artifacts from 23 SSW projects. SATD is most prevalent in pull requests and issue trackers, underscoring the value of multi-artifact analysis. Models trained on traditional SATD often miss *scientific debt*, emphasizing the need for its explicit detection in SSW. Practitioner validation confirmed that *scientific debt* is both recognizable and useful in practice. ***Conclusions:*** *Scientific debt* represents a unique form of SATD in SSW that that is not adequately captured by traditional categories and requires specialized identification and management. Our dataset, classification analysis, and practitioner validation results provide the first formal multi-artifact perspective on *scientific debt*, highlighting the need for tailored SATD detection approaches in SSW.

CCS Concepts

• **Software and its engineering** → **Maintaining software**; *Software evolution.*

Keywords

Technical Debt, SATD Classification, Deep Learning, Multitask Learning, Scientific Software, Scientific Debt

ACM acknowledges that this contribution was authored or co-authored by an employee, contractor, or affiliate of the United States government. As such, the United States government retains a nonexclusive, royalty-free right to publish or reproduce this article, or to allow others to do so, for government purposes only. Request permissions from owner/author(s).

PASC '26, Bern, Switzerland

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2734-4/2026/06
https://doi.org/10.1145/3815572.3815759

ACM Reference Format:
Eric L. Melin, Nasir U. Eisty, Gregory Watson, and Addi Malviya-Thakur. 2026. Multi-Artifact Analysis of Self-Admitted Technical Debt in Scientific Software. In *Platform for Advanced Scientific Computing Conference (PASC '26), June 29-July 01, 2026, Bern, Switzerland*. ACM, New York, NY, USA, 12 pages. https://doi.org/10.1145/3815572.3815759

1 Introduction

In software development, the saying "done is better than perfect" highlights the unavoidable balance between maintaining software quality and meeting deadlines. This mindset often results in compromises, where speed takes priority over robustness.

Technical debt (TD) [11] arises when practitioners choose expedient solutions rather than thorough and maintainable ones. Development teams often work under pressure due to limited resources, strict timelines, and evolving requirements [16]. Hence, developers may intentionally or unintentionally incur TD to expedite development, enabling teams to meet these deadlines, often at the cost of compromising the long-term integrity of the software system [34].

Similarly, self-admitted technical debt (SATD) [38], a subset of TD, refers to instances where developers explicitly acknowledge these compromises, generally documented in code comments, commit messages, pull requests, or issue tracker entries [29]. Based on the content of the acknowledged compromise, SATD instances can be categorized into numerous types [3].

While prior studies of SATD have largely examined general-purpose software systems, scientific software (SSW) presents unique challenges that warrant specialized attention [19, 37]. SSW supports critical domains such as climate modeling, energy systems, nuclear simulations, and biomedical research, where accuracy and reliability are essential for scientific discovery, reproducibility, policy decisions, and safety-critical outcomes. Unlike traditional software, shortcomings in scientific code may compromise not only maintainability but also the validity of scientific results. Pressures stemming from evolving scientific requirements, reliance on complex numerical methods, and the demand to rapidly disseminate findings create conditions for scientific debt [7], a specialized form of SATD. Consequently, dedicated approaches are needed to identify and categorize SATD in SSW to capture these unique risks and trade-offs.

Therefore, the goal of this study is to identify, categorize, and evaluate *scientific debt*, and to assess the extent to which existing SATD taxonomies capture the unique challenges and trade-offs inherent to SSW development. By examining *scientific debt* within

the context of critical domains such as climate modeling, energy systems, and biomedical research, this work seeks to enhance understanding of how domain-specific constraints, accuracy requirements, and evolving scientific objectives influence the manifestation and management of TD in SSW. To guide our study, we investigate the following research questions (RQs):

RQ1: How well does a multi-source SATD classifier detect and categorize SATD instances across SSW artifacts?
SSW artifacts, such as source code comments, commit messages, issue sections, and pull requests, capture SATD in different ways. We evaluate multiple candidate models to determine how well a classifier trained across these sources generalizes and select the strongest-performing model for downstream multi-artifact analysis.

RQ2: What is the prevalence of different SATD types across code comments, commit messages, pull requests, and issue tracking sections in SSW?
Different artifact types may surface distinct kinds and quantities of SATD. For example, *code/design debt* in code comments, *requirement debt* in issues, or *scientific debt* in discussions around results. Measuring their distribution is needed to guide both automated detection approaches and debt management practices.

RQ3: Can traditional SATD types capture *scientific debt* instances?
Traditional SATD categories, such as *code/design*, *documentation*, *test*, and *requirement debt*, were developed with industrial software in mind. SSW, however, introduces unique challenges, including domain-specific assumptions, computational accuracy concerns, and evolving scientific knowledge. Investigating whether existing SATD types adequately capture these cases, or whether *scientific debt* should be treated as a distinct category, is vital for extending SATD research to the scientific domain. Unlike RQ1, which focuses on model performance, RQ3 is designed to assess cross-class generalization. Specifically, how an existing SATD classifier behaves when confronted with scientific debt. To reflect this intent, we intentionally use a standard baseline model rather than the best-performing model identified in RQ1, allowing us to evaluate the limitations of traditional SATD types and classifiers in the context of SSW.

The key contributions and main findings of this study can be summarized as follows:

- *We identify and categorize scientific debt across multiple artifact types in SSW, providing the first formal multi-artifact analysis of this SATD subtype.*
- *We construct the first multi-artifact dataset for detecting scientific debt in SSW.*
- *We validate the practical relevance of scientific debt through a practitioner study with domain experts.*
- *We examine whether traditional SATD categories are sufficient to capture scientific debt or whether a dedicated category is necessary.*
- *We release a replication package, including all code and datasets, to support reproducibility and future research*[1].

[1]https://github.com/MelinHead225/PASC26-Replication-Package

2 Background and Related Works

2.1 SATD Taxonomy

SATD refers to instances where developers explicitly acknowledge the presence of TD within code comments or documentation. Early work of Potdar and Shihab [38] first formalized the concept of SATD, while Maldonado and Shihab [33] proposed an initial taxonomy consisting of five types of SATD. Subsequent research refined and expanded these categories [42]. Alves et al. [3] broadened the understanding of TD through a systematic mapping study that characterized debt types beyond self-admitted instances and across multiple software artifacts. Their ontology introduced thirteen categories of debt, organized by debt nature, offering a more comprehensive classification framework. In more recent SATD-focused work, researchers have converged on a taxonomy that emphasizes four primary types [29, 36, 44]: *code/design debt* (refer to suboptimal, temporary, or expedient implementation choices that degrade code quality or design structure), *documentation debt* (refer to improper/lacking documentation supporting that part of the program), *test debt* (refer to the need for implementation or improvement of the current tests), and *requirement debt* (express incompleteness of the method, class or program) [3, 33]. Notably, *code debt* and *design debt* are often treated as a unified *code/design debt* category due to their strong overlap in characteristics and implications for software maintenance [29, 44]. This refined taxonomy provides a structured foundation for both empirical analysis and automated detection of SATD, enabling researchers and practitioners to better identify, prioritize, and manage technical debt in software projects.

2.2 SATD Across Artifacts

Although early taxonomies were primarily constructed from code comments, recent research has demonstrated that SATD extends well beyond the source code. Developers frequently acknowledge debt across diverse artifacts, such as issue trackers, commit messages, pull requests, and architectural documentation. For instance, Li et al. [29] examined SATD across source code comments, commit messages, pull requests, and issues tracking sections, finding that SATD is distributed evenly across these four artifacts. Li et al. [28] focused on SATD in issue trackers, finding that the median time and average time to repay TD are 25.0 and 872.3 hours, respectively. Xavier et al. [48] also focused on issue tracker systems, uncovering that only 27% of SATD in issue tracker systems can be tracked back to source code comments. This expansion of SATD research highlights that debt is not limited to the implementation level but is distributed across the technical ecosystem of projects. Consequently, SATD taxonomies should account for domain-specific requirements to fully capture the spectrum of SATD in practice. Recognizing SATD across artifacts complements code-based detection and broadens opportunities for effective debt management.

2.3 Automated SATD Detection

Early efforts to detect SATD were largely grounded in keyword-driven heuristics [40, 41] and manual review processes [8, 39]. More recently, however, the field has shifted towards automation through machine learning and deep learning techniques [34].

A key milestone was introduced by Maldonado et al. [12], who applied natural language processing to automatically identify SATD in source code comments. Following this, a variety of traditional machine learning approaches were investigated, including Support Vector Machines [4, 29, 42, 47], Naïve Bayes [2, 23], and Random Forests [49, 50], trained on manually annotated SATD datasets. While computationally efficient, their effectiveness is often limited by domain-specific vocabulary and handcrafted features, consistently surpassed by modern transformer-based models.

While Gu et al. [18] represent closely related advances, our work differs in scope and objectives. Gu et al. extend SATD detection across multiple artifacts, yet their dataset and models do not include scientific software systems, nor do they consider the *scientific debt* category that is central to our study. In contrast, our work introduces the first multi-artifact SATD dataset drawn specifically from scientific software, incorporates scientific debt as a distinct SATD type, and evaluates its interpretability and utility with domain experts. Thus, our contribution complements prior SATD detection research while extending it to a domain and debt type not examined in earlier studies.

2.4 SSW and Scientific Debt

Scientific software (SSW) refers to software developed or used for scientific purposes [25]. It serves as the backbone of a wide range of disciplines and has become indispensable in modern research, including physics, biology, and medical analysis [6]. It supports the development, evaluation, and validation of scientific hypotheses by enabling sophisticated computations, data analyses, and simulations that would be infeasible or extremely time-consuming to carry out by hand [21].

Researchers frequently adopt software development practices that are heavily influenced by the immediate needs of their investigation and the rapid pace of scientific inquiry. Kelly [26] describes this as an "amethodical" approach, in which the urgency to produce research results often takes precedence over carefully planned, long-term software design. Prior studies of SSW development practices echo these observations: Pinto et al. [37] and Arvanitou et al. [6] report that scientific developers frequently prioritize immediate research needs over structured engineering processes, leading to ad-hoc development, irregular documentation, and evolving requirements.

Factors such as tight grant timelines, the pressure to publish promptly, budget limitations, evolving experimental methods, and, at times, limited attention to software longevity contribute to this short-term, pragmatic focus in SSW development. In SSW, correctness is a paramount concern. For example, in 2006, errors in scientific code contributed to the retraction of five high-profile scientific papers and the abandonment of numerous projects dependent on these findings [35]. Testing scientific software is particularly challenging due to the absence of test oracles, floating-point nondeterminism, and complex simulation behavior, as documented by Kanewala and Bieman [25].

The development of SSW, however, is fraught with unique challenges when compared to general open-source software. Developers of SSW are often domain scientists rather than formally trained computer scientists [5, 27], a finding reinforced by Arvanitou et al. [6], who reported that limited software engineering expertise contributes to inconsistent development practices and increased maintenance burden. Many projects are not originally designed to scale, but can grow substantially if initial trials prove successful [1]. Additionally, SSW is frequently used internally, either by its creator or by other members of the creator's research group [1]. According to Heaton [21], because developers vary significantly in background and objectives, techniques must be tailored to the SSW context.

At the heart of these challenges lies TD. In SSW, TD may manifest acutely due to the accumulation of suboptimal design or implementation choices that provide short-term benefits, incurring costs in maintenance, evolution, and reliability. A preliminary study on SATD in SSW by Awon [7] examined 28,680 code comments from 9 SSW repositories, where they defined a novel SATD called **Scientific Debt** as *the accumulation of sub-optimal scientific practices, assumptions, and inaccuracies within SSW that potentially compromise the validity, accuracy, and reliability of scientific results.* Unlike traditional TD, which primarily concerns code quality and architectural issues, *Scientific Debt* often arises from the unique intersection of scientific reasoning and software implementation. It emerges when developers encode incomplete theories, oversimplify models, or adopt provisional assumptions to make complex phenomena computationally tractable. These scientific shortcuts, while often necessary for progress, may accumulate over time and degrade the correctness or reproducibility of results.

Scientific Debt can manifest in several distinct forms, which Awon [7] identified as indicators:

- **Translation Challenges:** Difficulties in accurately representing scientific concepts within code, such as simplifying physical models or omitting interactions that are computationally expensive. For example, a comment in *Astropy* notes that a simplification of neutrino behavior is "not correct, but... a standard assumption," illustrating a trade-off between accuracy and practicality.
- **Assumptions:** Embedding approximations or unverified conditions due to missing data or limited understanding. For example, *CESM* developers note, "We assume here that new ice arrives at the surface with the same temperature as the surface," signaling a potential issue with energy conservation.
- **Missing Edge Cases:** Software that does not handle rare or boundary conditions in scientific computations, such as a *ROOT* comment stating, "This does not work for large molecules that span more than half of the box."
- **Computational Accuracy:** Issues where numerical precision or algorithmic simplifications lead to unreliable results, as seen in *GROMACS* where developers acknowledge that "for large systems, a float may not have enough precision."
- **New Scientific Findings:** Outdated constants or theories embedded in the software that no longer align with current scientific knowledge, necessitating continual updates to maintain scientific validity.

These examples reveal that *Scientific Debt* extends beyond code maintainability and is intertwined with the scientific process itself. Identifying and managing them is therefore crucial to ensure the reliability, reproducibility, and long-term evolution of SSW systems.

3 Dataset Creation

3.1 Overview

This section provides an overview of our approach for constructing a multi-source, high-quality dataset tailored to SATD identification in SSW by integrating and refining multiple data sources across software artifacts. The construction process involves four major phases: *source selection*, *label validation and annotation*, *iterative dataset expansion using pseudo-labeling and active learning*, and *semantic data cleansing*. Figure 1 provides an overview of the creation of the dataset.

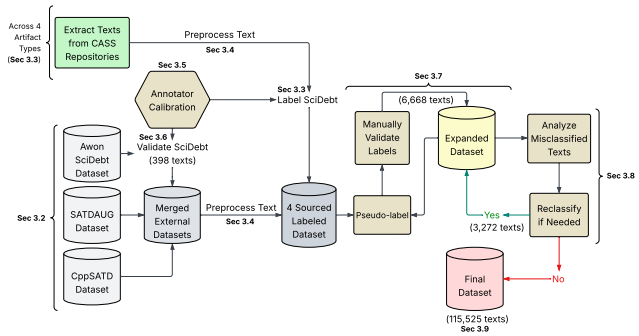


Figure 1: Dataset Creation Overview

3.2 Data Sources

We utilize three distinct initial datasets to provide coverage across different domains, languages, and SATD types:

1. **Augmented General Open Source Multi-Artifact Dataset (SATDAUG):** The SATDAUG dataset by Sutoyo and Capiluppi [44] is a multi-artifact SATD dataset containing labeled instances from code comments, commit messages, pull request sections, and issue tracker sections. It augments the dataset by Li et al. [29], which was sourced from 103 Apache projects on GitHub selected based on availability and complexity. The augmentation increases the representation of underrepresented SATD classes across artifacts, resulting in a more balanced dataset that improves ML/DL model performance. No *scientific debt* instances are present in this set.
2. **General Open Source C++ Comments Dataset (CppSATD):** The CppSATD dataset by Pham et al. [36] contains SATD-labeled code comments sourced from five C++ repositories selected based on popularity, activity, and application domain diversity. Since C++ is a dominant language in SSW, this dataset helps ensure model generalization to C++-specific syntax and terminology. Like SATDAUG, it does not contain scientific debt instances.
3. **Scientific Software Dataset (Awon):** This dataset is from Awon [7]. It is comprised of 1,301 scientific debt comments from nine open-source SSW projects supporting the specific SATD type to SSW *scientific debt*. We manually validate and curate this dataset (see Section 3.6).

3.3 Initial Scientific Software Annotation

We selected a set of nine SSW repositories from the U.S. Department of Energy (DOE) Consortium for the Advancement of Scientific Software (CASS)[2] for additional annotation to expand the number of scientific debt examples and increase domain diversity. Because there is no standardized set of criteria for selecting repositories in SATD research, we followed practices consistent with prior software engineering studies that rely on indicators of repository maturity, sustained activity, and community engagement [7, 9, 30]. To select a subset of CASS repositories to label, we selected projects that were:

- Publicly available on Github.
- Had been contributed to in the last four months.
- Had at least 10k commits.
- Had at least 20 contributors.
- Were at least two years old.
- Had at least 40 stars.

We used these guidelines to ensure that the projects were publicly available, actively maintained, sufficiently complex, relevant, and sufficiently mature. CASS projects span five areas: data and visualization, development tools, mathematical libraries, programming models and runtimes, and software ecosystem and delivery. To ensure domain breadth, we selected at least one repository for each area. We cloned each of the nine selected CASS projects' main branches on June 18th, 2025, for data extraction.

Using a keyword-based heuristic search (see Footnote 1), we identified potentially relevant comments across the four artifact types, emphasizing terms linked to uncertainty, assumptions, translation, accuracy, edge cases, or evolving scientific findings.

Using this first batch of potential *scientific debt* texts, we manually labeled instances that matched the *scientific debt* definition to create an initial seed set for the first iteration of pseudo-labeling for each artifact type. Texts that did not meet the *scientific debt* criteria were labeled according to previously established SATD classes from the literature [29, 36, 44]. By annotating instances across all supported classes from the CASS projects, we aimed to reduce domain overfitting.

3.4 Data Preprocessing

We applied a series of preprocessing and data cleansing procedures to ensure consistency across heterogeneous software artifacts. Following prior work [13, 23], we normalized all textual data by converting multi-line comments into single lines, lowercasing all text, and removing non-alphabetic characters, while preserving question and exclamation marks since they may signal SATD. We removed extra whitespace, empty lines, and excluded comments containing licensing information (e.g., license, copyright, distributed under). We also removed duplicate entries across all types of artifacts to prevent bias in downstream classification.

Because our datasets span multiple artifact types, preprocessing was tailored to each source:

- We standardized source code comments across multiple programming languages (Python, C/C++, Fortran, Java, Shell, CMake,

[2] https://cass.community/software/

MATLAB, Rouge) by removing language-specific comment delimiters. We merged consecutive full-line comments, while retaining inline comments following code fragments as separate units. Commit messages only required following the basic preprocessing procedure [23].
- We segmented pull requests and issues tracking sections into titles, descriptions, and individual comments, following the multi-artifact design proposed by Li et al. [29]. Treating these sections separately was important because different SATD cues often occur in different parts of the artifact: titles capture intent, descriptions provide rationale or design trade-offs, and comments reflect discussion and evolving decisions. We normalized each section consistently, removed bot-generated comments (manually identified by usernames), and discarded pull requests or issues with no usable content.
- To verify the correctness of the preprocessing pipeline, we manually inspected a random sample of 100 cleaned instances for each artifact type. All inspected instances were satisfactory and required no modifications to the pipeline. We selected 100 examples per artifact type as a pragmatic balance between coverage and feasibility. The sample size was large enough to expose any systematic preprocessing issues while still allowing thorough manual verification. This sanity-check confirmed that artifacts remained coherent textual units and that key structural cues and cross-references were preserved for downstream SATD analysis.

3.5 Labeler Calibration and Agreement

To ensure consistent annotation quality, we first conducted a calibration exercise following the procedure used in [29]. Using a predefined annotation guideline document (see Footnote 1) for classifying SATD data points, we prepared a set of 50 unlabeled samples for each of the four types of artifacts from the selected SSW repositories (see Section 3.3), which were independently labeled by two of the authors. We then measured the agreement between annotators using Cohen's Kappa (see Equation 1), ensuring the mutual understanding of all supported classes, including *scientific debt*. In case of disagreement, we discussed our differences and performed another iteration of calibration. The results of our calibration can be seen in Table 1.

Cohen's Kappa Formula

$$\kappa = \frac{p_o - p_e}{1 - p_e} \tag{1}$$

where p_o is the observed proportion of agreement and p_e is the expected agreement by chance.

Table 1: Cohen's Kappa Results for Annotation Calibration Across Data Sources

Data Source	Agreement (out of 50)	Cohen's κ
Comments	49/50	0.953
Commits	49/50	0.957
Issues	49/50	0.960
Pull Requests	46/50	0.843
Overall Combined	193/200	0.926

The resulting Cohen's Kappa scores indicate "almost perfect" agreement across all the calibration datasets.

3.6 Manual Validation of Scientific Debt Annotations

We manually validated all labeled instances of the *scientific debt* class from the Awon dataset to ensure alignment with the domain-specific definition of *scientific debt*, as defined in Section 2.4 and our annotation guideline document. During validation, we confirmed instances of scientific debt aligned with comments expressing the accumulation of suboptimal scientific practices, assumptions, or inaccuracies within SSW that could compromise the validity, accuracy, or reliability of scientific results. We also removed ambiguous comments that could be interpreted as another SATD class to minimize feature overlap between *scientific debt* and existing SATD categories, which led to the removal of a substantial number of labeled instances. This process resulted in the candidate *scientific debt* comments being reduced from 1301 to 398. This filtering ensured that the *scientific debt* class in our dataset consisted only of clearly grounded, domain-relevant examples, improving label quality and model reliability.

3.7 Iterative Dataset Expansion via Pseudo-Labeling and Active Learning

To scale the *scientific debt* dataset beyond the manually curated seed set, we implemented an *iterative expansion strategy* that combines **pseudo-labeling** and **active learning**. This approach enabled efficient discovery of new *scientific debt* instances from candidate CASS projects while maintaining label quality and minimizing manual search effort.

The process we employed was as follows:

1. **Initial Model Training:** We fine-tuned BERT-base [14] a transformer-based language model pre-trained on large corpora to learn general linguistic representations on the combined dataset, which included:
   - The manually validated *scientific debt* examples from the Awon dataset (Section 3.6).
   - The manually annotated samples from CASS projects (Section 3.3).
   - The general open-source multi-artifact SATDAUG dataset (non-scientific).
   - The CppSATD C++ code comment dataset (non-scientific).
2. **Prediction on Unlabeled Data:** The trained model was applied to the remaining set of *unlabeled artifacts* (comments, pull requests, issues, and commit messages) from the candidate SSW repositories. This step yielded class predictions and associated softmax probabilities.
3. **Manual Annotation of Model Predictions:** We selected a subset of the model's predictions on unlabeled data points for *manual validation* using the softmax probabilities of the classifications:
   - High-confidence predictions likely to represent *scientific debt* (to expand coverage), and eliminate misclassifications of high confidence due to potential domain shift originating from the multiple datasets utilized.
   - Low-confidence or borderline *scientific debt* cases (to support active learning and identify weak decision boundaries).
   - A curated set of texts supporting all SATD classes from CASS repositories to reduce domain overfitting.

We manually reviewed and annotated these samples using our established SATD classification guidelines (see Footnote 1).

4. **Dataset Update and Retraining:** We appended the verified annotations to the dataset, and retrained the classifier using the expanded dataset. This process was repeated across approximately 10 rounds, after analyzing approximately 2,000 instances, incrementally improving model performance and increasing the dataset size. Overfitting was prevented by fine-tuning the original pretrained model rather than the finetuned model on the expanded dataset on each iteration.

This semi-supervised, human-in-the-loop approach supported efficient dataset growth of all classes from CASS projects while preserving label quality and ensuring that our model did not suffer from domain bias by building a label set across all supported classes. In total, this process resulted in an additional 664 *scientific debt* labeled texts, and 5,616 miscellaneous labeled texts from CASS repositories to minimize domain bias.

3.8 Dataset Validation

Given the integration of multiple datasets in this study, we conducted approximately 10 total iterations of stratified 5-fold validation, evaluating a fine-tuned BERT-base model on each individual artifact dataset as well as on the merged master dataset containing all four artifact sources. We saved misclassified instances for manual inspection. This manual review of misclassified examples across folds enabled semantic dataset cleansing, allowing us to relabel possible instances of *scientific debt* from datasets that did not initially support this SATD type.

Moreover, although each dataset annotated the same SATD types, we noticed discrepancies due to differences in annotators' judgments. Many imported labeled texts did not align with our annotation standards, prompting us to standardize ambiguous instances across several iterations. This semantic data cleansing process led to manually relabeling 3,272 texts.

To assess the quality of the finalized merged dataset, similar to Li et. al [29] we randomly selected a sample larger than the statically significant sample size for our dataset with 95% confidence with a 5% margin of error (i.e., 384 samples). The first author then independently annotated this sample classifying texts as SATD or non-SATD, and the Cohen's kappa coefficient was calculated to evaluate consistency and reliability of the final dataset. The results of this evaluation yields a Cohen's kappa of 0.734 indicating "substantial agreement" with dataset sample with an observed agreement of 0.893 over all labeled texts.

3.9 Final Dataset Summary

The resulting dataset is a *multi-source, multi-domain corpus* curated for the identification and classification of SATD in SSW. It integrates:

- **Four artifact types:** code comments, commit messages, issue tracking sections, and pull requests sections.
- **Three source domains:**

(1) General open-source repositories (augmented for increased class balance),
(2) C++ projects (for programming language representation),
(3) Scientific software repositories (target domain).

- **An additional SATD class:** *scientific debt*, rigorously defined and validated for this work.
- **Hybrid labeling methodology:** combining manual annotation, pseudo-labeling, active learning, and semantic data cleansing.

Key dataset statistics across code comments (CC), commit messages (CM), issues sections (IS), and pull request sections (PR) are summarized in Table 2. Definitions for each SATD type can be found in Section 2.1 and 2.4.

Table 2: Labeled SATD Class Distribution Across Artifact Types

SATD Class	CC	CM	IS	PR	Total
Requirement Debt	4,243	615	2,042	504	7,404
Code/Design Debt	7,915	247	2,315	643	11,120
Documentation Debt	1,980	616	1,958	517	5,071
Test Debt	2,765	757	2,066	592	6,180
Scientific Debt	741	56	164	148	1,109
Non-SATD	54,861	5,870	18,724	5,185	84,640
Total	72,505	8,161	27,269	7,589	115,524

This dataset represents the **first multi-artifact SATD dataset** specifically designed to support the **identification of SATD in SSW**. It enables reproducible benchmarking for classification models and lays the foundation for future research into SATD in SSW.

4 Practitioner Validation

We conducted a Google Form-based validation with seven scientists and researchers to assess whether *scientific debt* is understandable and useful in real workflows. Participants were given a brief introduction to SATD and *scientific debt*, along with illustrative examples from our dataset.

Participants were recruited through professional contacts within the scientific software community at U.S. national laboratories. We directly invited experienced practitioners who actively develop or maintain scientific software.

Participants represented a diverse set of scientific domains, including Supporting I/O Library and HDF5, Programming Models and Scalable Algorithms, Computer Science and Neutron Sciences, Research Software Engineering, Computer and Information Science, High Performance Computing, and Programming Systems and Scientific Libraries for HPC. On average, participants reported 18 years of relevant experience.

Following the introduction, participants were presented with text snippets from our dataset, which contained four randomly selected *scientific debt* text snippets and asked to respond to two questions for each snippet:

1. **Classification Judgment:** Does this text demonstrate *scientific debt* as defined?
2. **Practical Usefulness:** How useful is the label of *scientific debt* for identifying suboptimal scientific practices? Participants rated usefulness on a Likert scale from 1 (Strongly Disagree) to 5 (Strongly Agree).

For snippets labeled as *scientific debt*, participants agreed with the label 78.6% of the time, while the remaining 21.4% selected "unsure". Notably, no participant explicitly disagreed with any annotation suggesting that the annotation guidelines were generally

interpretable and aligned with practitioner expectations. However, the lack of explicit disagreement should be interpreted cautiously, as it does not necessarily imply complete consensus on all individual labels.

Regarding the usefulness of the label to identify suboptimal scientific practices, the participants provided an average Likert score of 4.03, indicating the label is perceived as useful in practice.

Participants also provided open-ended feedback on the usefulness and potential value of identifying *scientific debt*. Several participants noted that shorter snippets were easier to interpret and assess actionable assumptions or potential problems, while longer snippets were more challenging to evaluate. Suggestions included paraphrasing snippets to improve readability, rather than providing the raw text, and offering more granular categories of *scientific debt* instead of just a binary *scientific debt*/*non-debt* classification utilized for validation. Participants generally found the examples interesting and expressed a desire for further exploration of *scientific debt* in real workflows, indicating both conceptual relevance and practical interest in the label.

These validation findings support the quantitative results of RQs, showing that not only does our multi-source SATD classifier perform well across *scientific debt* instances, but the concept of *scientific debt* is also recognizable and deemed practically useful by domain experts, reinforcing the relevance of our automated classification approach in real-world workflows.

5 Methodology

To answer the proposed RQs, we utilized the multi-artifact dataset constructed in Section 3. We annotated each labeled data point into one of four artifact types: code comments (CC), commit messages (CM), issue tracker sections (IS), and pull request sections (PR) and one of six SATD classes: *requirement debt, code/design debt, documentation debt, test debt, scientific debt, and non-debt*. Given the lack of a universal state-of-the-art classifier for heterogeneous SATD identification, we fine-tuned multiple transformer-based language models to determine which was most effective for addressing the proposed research questions.

We evaluated the following transformer-based models, including general-purpose (GP) LLMs, domain-specific (DS) models for scientific text, and code-oriented (CO) models:

- GP: **meta-llama/Llama-2-7b-hf** [46]
- GP: **mistralai/Mistral-7B-Instruct-v0.3** [24]
- GP: **tiiuae/Falcon3-3B-Instruct** [45]
- GP: **meta-llama/Llama-3.2-1B** [17]
- GP: **google-bert/bert-base-uncased** [14]
- GP: **FacebookAI/roberta-base** [31]
- GP: **microsoft/deberta-v3-base** [20]
- DS: **allenai/scibert_scivocab_uncased** [10]
- CO: **microsoft/codebert-base** [15]

5.1 Experimental Setup

All models were fine-tuned using the first author's institutional high-performance computing (HPC) cluster equipped with NVIDIA L40 GPUs, each with 48GB of GDDR6 memory. For each model, we employed a grid search to find the model's optimal hyperparameter settings for fine-tuning:

- *Learning Rates*: $\{1 \times 10^{-5}, 5 \times 10^{-5}, 1 \times 10^{-4}\}$
- *Weight Decays*: $\{0.0, 0.01, 0.1\}$
- *Batch Sizes*: chosen as the maximum size supported by GPU memory for each model, selected from $\{256, 128, 64, 32, 16, 8\}$

We maximized batch sizes to stabilize gradients and improve training efficiency. We adopted seven-billion-parameter models using Low-Rank Adaptation (LoRA) [22] and 4-bit quantization to fit within compute constraints.

To handle heterogeneous artifacts, we adopted a multi-task learning design, where each artifact type was treated as a distinct task with its own classification head. A shared transformer encoder produced contextual embeddings that were pooled from the final hidden states and routed to the appropriate task-specific head. This design enabled shared learning across artifacts while capturing task-specific nuances critical for SATD detection.

We trained and evaluated the models using stratified 3-fold cross-validation, preserving class distributions across folds. Within each fold, we trained the models for a minimum of three epochs using the AdamW optimizer [32] with linear learning rate scheduling. We stopped the training early if the validation loss did not improve for two consecutive epochs, preventing overfitting and reducing unnecessary computation.

We assessed model performance using per-class precision, recall, and F1 scores. Precision reflects how often predicted instances of a SATD category are correct, while recall measures how effectively true instances are identified; the F1 score balances these two measures. Because SATD categories are highly imbalanced, we emphasize macro-averaged F1, which weights all classes equally rather than being dominated by majority classes (non_debt). We also computed fold-level accuracies to capture variance across splits and aggregated metrics across folds to provide stable performance estimates, allowing us to quantitatively answer our proposed RQs.

5.2 Model Selection

Among the evaluated models, **tiiuae/Falcon3-3B-Instruct** demonstrated the strongest performance across classification tasks. In particular, Falcon achieved the highest macro-F1 score—the most appropriate metric for SATD detection given the substantial class imbalance—marginally outperforming the next-best model (meta-llama/Llama-3.2-1B) by a margin of 0.002.

Larger 7B-scale models did not outperform Falcon, likely due to the low-rank adaptation (LoRA) required to fit these models within our GPU memory constraints. The additional parameter efficiency constraints imposed by LoRA may have reduced their effective capacity relative to the fully fine-tuned models up to 3B, resulting in weaker downstream performance. Consequently, we selected Falcon as the primary inference model for answering RQ1 and RQ2.

We saved the fine-tuned Falcon model, along with its tokenizer and task-specific classification heads, locally and uploaded them to HuggingFace[3] for reproducibility and downstream application.

6 Results

Using the methodology and dataset described in the prior sections, we produced a number of results and findings presented here, organized by the previously defined research questions.

[3] https://huggingface.co/MelinHead225/multitask-falcon-satd2

6.1 RQ1: How well does a multi-source SATD classifier detect and categorize SATD instances across SSW artifacts?

Table 3 summarizes the cross-validation performance of our best-performing fine-tuned model, Falcon-3B-Instruct, across all SATD classes. The model demonstrates strong predictive capability, achieving an average accuracy of 0.9159 and an average macro F1-score of 0.8255. Performance is particularly high for *documentation debt* (F1-score 0.9277), *test debt* (0.9129), and *non-debt* (0.9540) classes.

Table 3: Cross-Validation Performance Summary

Class / Metric	Precision	Recall	F1-score
requirement_debt	0.8481	0.8062	0.8266
code/design_debt	0.7160	0.6784	0.6966
documentation_debt	0.9419	0.9140	0.9277
test_debt	0.9303	0.8961	0.9129
scientific_debt	0.7422	0.5564	0.6353
non_debt	0.9451	0.9630	0.9540

Our domain-specific *scientific debt*, exhibits comparatively lower performance with an F1-score of 0.6353. This is likely due to the extreme sparsity of this class in the training data, as scientific debt represents less than 1% of the dataset (0.959%). Further, performance may be influenced by including external datasets that were not originally designed to support this debt class. Although we semantically cleansed 3,272 texts from these external sources, it was not feasible to apply the same process to the entire 115k-text dataset.

We also observed that adopting a multi-head (task-specific) classification design yields a modest but consistent improvement of approximately 0.02 in macro F1 compared to single-head models. This suggests that task-specific heads effectively leverage artifact-specific characteristics while benefiting from shared transformer representations.

The model demonstrated balanced precision and recall across most classes, indicating consistent detection without excessive false positives or negatives. These results suggest that a multi-task, multi-artifact approach is effective for identifying and categorizing SATD instances across heterogeneous software artifacts.

RQ1 Findings: The multi-source SATD classifier demonstrates strong overall performance, achieving an average accuracy of 0.9159 and an average macro F1-score of 0.8255. The model performs particularly well on *documentation debt*, *test debt*, and *non-debt* classes, while *scientific debt* exhibits lower performance (F1 = 0.6353) due to class sparsity and inclusion of external datasets not originally supporting the class. Using a multi-head (task-specific) classification design improves overall macro F1 by approximately 0.02 compared to a single-head model, indicating the benefit of artifact-specific heads.

6.2 RQ2: What is the prevalence of different SATD types across code comments, commit messages, pull requests, and issue tracking sections in SSW?

To investigate the prevalence of SATD in SSW, we selected an additional 23 open-source CASS projects distinct from those used during training to be analyzed by our SATD identification pipeline. These projects are highly similar to the original 9 training projects in both scientific domain and development context: all are U.S. Department of Energy supported scientific software systems, built for high-performance computing workflows, and developed under similar programming practices, research objectives, and community standards. The extraction of the four artifacts: code comments, commit messages, pull requests, and issue tracking sections resulted in 900,358 texts to be classified.

The distribution of SATD types across artifact sources in Table 4 reveals distinct patterns in how TD manifests within SSW:

- **Code comments** are overwhelmingly *non-debt* (98.04%), but they still contain small proportions of *code/design debt* (1.30%) and *scientific debt* (0.17%). This indicates that developers occasionally document scientific assumptions, approximations, or unresolved computational challenges directly in the code. *Documentation debt* is nearly absent (0.01%), reflecting that code comments in SSW are primarily used for functional explanations rather than high-level design or documentation discussions.
- **Commit messages** exhibit moderate prevalence of SATD (5.98%), with *documentation debt* (2.01%) and *test debt* (1.55%) being the most common. *Scientific debt* is very rare (0.06%), suggesting that scientific concerns are seldom captured in commit metadata and instead are more likely embedded in code comments or issue discussions. Commit messages appear to mainly capture procedural or logistical aspects of changes.
- The **Issue tracking sections** show a higher prevalence of SATD (9.62%), with *code/design debt* (4.10%) being dominant, followed by *documentation debt* (2.36%) and *scientific debt* (1.22%). This pattern suggests that SSW developers use issues not only to discuss design and architectural improvements but also to document unresolved scientific challenges or potential inaccuracies that could affect the validity of the simulation or the experimental results.
- **Pull request sections** have the highest SATD prevalence (12.00%), with *code/design debt* (6.04%) and *documentation debt* (3.37%) leading. *Scientific debt* is less frequent (0.36%) but still present, reflecting the collaborative nature of pull requests, where reviewers occasionally comment on scientific correctness or potential computational approximations. Pull requests thus serve as a critical artifact for both code quality and the verification of scientific fidelity in SSW.

Across all artifacts, *requirement debt* remains consistently low (0.22%–1.11%), indicating that requirement-related issues are infrequently documented as explicit TD within scientific contexts.

Table 4: Distribution of predicted SATD types across artifact sources in 23 CASS projects. Percentages indicate the proportion of each SATD type within the artifact source.

Artifact Source	Code/Design	Documentation	Requirement	Test	Scientific	Non-Debt
Code Comments	5,660 (1.30%)	47 (0.01%)	945 (0.22%)	1,137 (0.26%)	743 (0.17%)	427,496 (98.04%)
Commit Messages	4,205 (1.99%)	4,234 (2.01%)	761 (0.36%)	3,266 (1.55%)	135 (0.06%)	198,274 (94.02%)
Issue Sections	4,109 (4.10%)	2,362 (2.36%)	1,109 (1.11%)	833 (0.83%)	1,227 (1.22%)	90,536 (90.38%)
Pull Request Sections	9,239 (6.04%)	5,152 (3.37%)	1,068 (0.70%)	2,362 (1.54%)	546 (0.36%)	134,701 (88.00%)

RQ2 Findings: In SSW, pull request sections exhibit the highest SATD prevalence (12.00%), primarily in *code/design debt* (6.04%) and *documentation debt* (3.37%). Issue tracking sections are also important for capturing *scientific debt* (1.22%) along with *code/design debt* (4.10%) and *documentation debt* (2.36%). Code comments primarily contain *non-debt* content, though they do capture scientific assumptions and computational approximations (0.17%). Commit messages show moderate SATD prevalence (5.98%), mainly in *documentation debt* (2.01%) and *test debt* (1.55%). *Requirement debt* is consistently low across all artifact types.

6.3 RQ3: Can traditional SATD types capture scientific debt instances?

We investigate whether a model trained exclusively on traditional SATD types (i.e., *requirement debt*, *code/design debt*, *documentation debt*, *test debt*, and *non-debt*) can identify instances of *scientific debt*, even if misclassified as a traditional SATD type. This analysis allows us to evaluate the extent to which explicitly modeling *scientific debt* uncovers new, previously unrecognized SATD in SSW.

Because the goal of RQ3 is to assess cross-class generalization rather than optimize performance, we deliberately use a standard baseline model (BERT-base) instead of the best-performing classifier identified in RQ1. This choice allows us to evaluate how existing SATD approaches could behave when confronted with scientific debt, a category absent from prior taxonomies.

To carry out this investigation, we fine-tuned BERT-base [14] on our final dataset (Section 3.9), excluding all samples of *scientific debt* from the training set. The training dataset, therefore, contained only five classes, while the evaluation set consisted solely of samples labeled as *scientific debt*.

We implemented a standard PyTorch training loop with the following setup: a maximum sequence length of 128 tokens, a batch size of 128, a learning rate of 0.0001, a weight decay of 0.1, and training for 5 epochs based on a grid search of the best parameters. We utilized the AdamW optimizer [32] and a linear learning rate scheduler to update model parameters. Model predictions were then obtained by selecting the class with the highest output probability.

The model predictions for *scientific debt* samples is summarized in Table 5. Most instances (74.4%) were classified as *non-debt*, indicating that *scientific debt* reveals a substantial number of previously undetectable issues in the software. The remaining 25.6% (observable in Table 5) instances were labeled as traditional SATD types, suggesting that some features of *scientific debt* overlap with existing SATD categories.

Table 5: Inference on *scientific debt* using a model trained on traditional SATD types.

Predicted Label	Count
non-debt	751
code/design debt	256
requirement debt	72
test debt	28
documentation debt	2
Total	1,009

To illustrate the limitations of traditional SATD classes, below are representative *scientific debt* instances that were misclassified as *non-debt* when using only traditional SATD categories:

- "surprisingly poor tolerance because sign change constraint limits correction this may be worth exploring further"
- "initialize f ground choose a special non physical value to be overwritten in all cells with ice todo make sure the special value isn t used in any calculations"
- "cap the staggered effective pressure at x and x overburden pressure to avoid strange values going to the friction laws todo remove capping for coulomb cases since the effective pressure is already capped above by capping f?"

RQ3 Findings: These results demonstrate that relying solely on traditional SATD categories fails to capture domain-specific SATD in SSW as the majority of manually labeled *scientific debt* instances were misclassified as *non-debt*. This underscores the need to include *scientific debt* as a distinct category in SATD management for SSW. By doing so, practitioners and researchers can more accurately identify, prioritize, and address issues that are unique to scientific computation, improving both software quality and the reliability of scientific results.

7 Discussion

Our study provides several important insights into the identification, prevalence, and domain-specific nature of SATD in SSW.

7.1 Multi-Artifact SATD Detection

Our first-of-its-kind, multi-source SATD classifier tailored towards SSW demonstrates strong overall performance (RQ1), particularly for *documentation debt*, *test debt*, and *non-debt* categories. The comparatively low performance for *scientific debt* (F1 = 0.6353) is likely a result of class sparsity and the use of external datasets that are not fully aligned with the contexts of SSW. Despite this, the model reliably captures general SATD patterns (F1 = 0.8255) across artifact sources, and the multi-head (task-specific) design improves

macro F1 by approximately 0.02, highlighting the benefit of artifact-specific adaptations.

To the best of our knowledge, no prior work has explored SATD detection across multiple artifact types in scientific software, nor are there published datasets that support such cross-artifact evaluation. Most existing SATD identification studies focus exclusively on code comments and report only F1-scores rather than macro-F1, which is more appropriate for highly imbalanced datasets. Existing fine-tuning approaches [18, 43] report F1-scores in the mid 0.80s on code comment SATD identification. In comparison, our macro-F1 of 0.8255 across heterogeneous artifact types demonstrates competitive performance relative to existing SATD approaches and highlights the value of our dataset for supporting broader SATD research in SSW.

7.2 SATD Prevalence Across Artifacts

The RQ2 results indicate that in SSW, pull request sections and issue tracker sections are the richest sources of SATD in SSW, with pull requests exhibiting the highest overall prevalence (12.0%) and issue tracking sections closely following (9.62%). Code comments contain predominantly *non-debt* content (98.04%) but do provide insight into scientific assumptions and unresolved computational issues. Commit messages, while moderately rich in SATD (5.98%), mainly capture procedural or logistical notes. In particular, *scientific debt*, though less frequent than *code/design debt* or *documentation debt*, is identifiable in issue trackers (1.22%) and pull requests (0.36%), highlighting its relevance in collaborative and review contexts.

7.3 Limitations of Traditional SATD Categories

RQ3 demonstrates that models trained exclusively on traditional SATD types fail to capture domain-specific issues in SSW, with the majority of *scientific debt* instances misclassified as *non-debt*. This finding underscores the need for domain-specific approaches when managing SATD. Although we observe some overlap of *scientific debt* (25.6%) with traditional SATD classes in this exclusion experiment, these results show that the sole reliance on traditional categories leaves a substantial portion of critical domain-specific debt undetected.

7.4 Practitioner Validation

To further assess the interpretability and perceived usefulness of *scientific debt*, we conducted a study validation with seven scientific software scientists and developers from diverse scientific domains, averaging 18 years of experience. Participants were introduced to the concept and shown labeled text snippets from our dataset. The agreement with our *scientific debt* labels was high (78.6%), with the remaining 21.4% marked as "unsure" rather than incorrect, indicating conceptual alignment. The participants also rated the usefulness of the *scientific debt* label highly (average Likert score = 4.03/5), confirming that the concept resonates with the practices of real-world SSW development. Open-ended feedback suggested that shorter snippets improve interpretability and that *scientific debt* subcategories could offer more actionable insight. These qualitative results complement our quantitative findings, demonstrating that *scientific debt* is both recognizable and practically meaningful to domain experts.

7.5 Implications for SSW Development

The results suggest that monitoring multiple artifact sources is essential to identify and mitigate TD in SSW. Pull request sections and issue tracking sections are particularly valuable for uncovering both conventional and scientific SATD. Incorporating *scientific debt* as a distinct category in SATD management can help prioritize computational accuracy, assumptions, and outdated translation challenges that may otherwise go unnoticed. The strong agreement between practitioners observed in the validation further validates the practical relevance of this concept and supports its integration into future SATD detection frameworks.

8 Threats to Validity

Several factors may affect the validity of these findings:

- **Construct Validity:** Defining and labeling *scientific debt* involves inherent subjectivity. Despite detailed annotation guidelines, interpretations may vary, and explicit admissions were emphasized, potentially missing unacknowledged inaccuracies or nuances present in the full code context.
- **Internal Validity:** Our use of multiple datasets, including external sources without *scientific debt*, introduces potential label noise and context mismatch. In addition, one incorporated dataset lacked project-level identifiers, so we could not reliably separate samples by project, preventing true cross-project validation. Treating all artifacts as coming from a single project risks subtle leakage and reduces the validity of cross-project performance estimates. Stratified k-fold cross-validation mitigates class imbalance, but fine-tuned transformers may still overfit dataset-specific linguistic patterns despite early stopping.
- **External Validity:** The study focuses on 23 open-source CASS projects, which may not represent all SSW domains or private repositories. Language diversity, community culture, and temporal changes in SSW practices may limit generalization to other contexts.
- **Conclusion Validity:** Imbalanced distributions of SATD types and artifacts reduce statistical confidence for rare categories, making metrics like accuracy and F1-score sensitive to small fluctuations. Larger, more balanced datasets would help confirm the robustness of these findings.

9 Conclusion

This study presents the first multi-artifact analysis of SATD in SSW, with a particular focus on domain-specific *scientific debt*. The key takeaways are as follows:

1. Multi-artifact SATD classifiers effectively detect and categorize self-admitted technical debt across heterogeneous SSW artifacts, although class sparsity remains a challenge for domain-specific debt types.
2. Pull requests and issue tracking sections are primary artifacts for identifying SATD, including scientific considerations, while code comments and commit messages capture complementary, though less frequent, instances.
3. Traditional SATD categories are insufficient to capture domain-specific *scientific debt*; explicitly modeling *scientific debt* is necessary for comprehensive SATD management in scientific software.

4. Expert feedback from scientists and software developers confirms that the concept of *scientific debt* is both understandable and useful in real scientific workflows, reinforcing its relevance for future SSW maintenance practices.

By highlighting the presence and patterns of *scientific debt*, this work enables practitioners and researchers to better prioritize improvements, maintain scientific accuracy, and improve the reliability of computational results. Future work should extend the analysis to additional SSW domains, refine subcategories of *scientific debt* based on practitioner feedback, explore automated mitigation strategies, explore SSW-specific SATD prioritization approaches, and explore management strategies.

10 Data Availability

The dataset, code, and annotation guidelines are available at https://github.com/MelinHead225/PASC26-Replication-Package. The trained model is publicly available on HuggingFace at https://huggingface.co/MelinHead225/multitask-falcon-satd2.

11 Acknowledgments

This material is based upon work supported by the U.S. Department of Energy, Office of Science, Office of Workforce Development for Teachers and Scientists, Office of Science Graduate Student Research (SCGSR) program. The SCGSR program is administered by the Oak Ridge Institute for Science and Education (ORISE) for the DOE. ORISE is managed by ORAU under contract number DESC0014664. We acknowledge the support of the Consortium for Open-Source Research Software Advancement (CORSA), a project supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Next-Generation Sci- entific Software Technologies program, under contract number DE-AC05-00OR22725. All opinions expressed in this paper are the author's and do not necessarily reflect the policies and views of DOE, ORAU, or ORISE.

The authors also gratefully acknowledge the practitioner participants whose time, expertise, and insights were invaluable in validating and strengthening this study.

References

[1] K. S. Ackroyd, S. H. Kinder, G. R. Mant, M. C. Miller, C. A. Ramsdale, and P. C. Stephenson. Scientific software development at a research facility. *IEEE software*, 25(4):44, 2008.

[2] A. Alhefdhi, H. K. Dam, Y. S. Nugroho, H. Hata, T. Ishio, and A. Ghose. A framework for conditional statement technical debt identification and description. *Automated Software Engineering*, 29(2):60, 2022.

[3] N. S. Alves, L. F. Ribeiro, V. Caires, T. S. Mendes, and R. O. Spínola. Towards an ontology of terms on technical debt. In *2014 sixth international workshop on managing technical debt*, pages 1–7. IEEE, 2014.

[4] F. Arcelli Fontana, M. V. Mäntylä, M. Zanoni, and A. Marino. Comparing and experimenting machine learning techniques for code smell detection. *Empirical Software Engineering*, 21:1143–1191, 2016.

[5] D. C. Arnold and J. J. Dongarra. Developing an architecture to support the implementation and development of scientific computing applications. In *The Architecture of Scientific Software*, 2000.

[6] E.-M. Arvanitou, A. Ampatzoglou, A. Chatzigeorgiou, and J. C. Carver. Software engineering practices for scientific software development: A systematic mapping study. *Journal of Systems and Software*, 172:110848, 2021.

[7] A. M. Awon. *Self-Admitted Scientific Debt: Navigating Cross-Domain Challenges in Scientific Software*. PhD thesis, University of Victoria, 2024.

[8] G. Bavota and B. Russo. A large-scale empirical study on self-admitted technical debt. In *Proceedings of the 13th International Conference on Mining Software Repositories*, MSR '16, page 315–326, New York, NY, USA, 2016. Association for Computing Machinery.

[9] G. Bavota and B. Russo. A large-scale empirical study on self-admitted technical debt. In *Proceedings of the 13th international conference on mining software repositories*, pages 315–326, 2016.

[10] I. Beltagy, K. Lo, and A. Cohan. Scibert: A pretrained language model for scientific text. In *EMNLP*. Association for Computational Linguistics, 2019.

[11] W. Cunningham. The wycash portfolio management system. *ACM Sigplan Oops Messenger*, 4(2):29–30, 1992.

[12] E. da S. Maldonado, E. Shihab, and N. Tsantalis. Using natural language processing to automatically detect self-admitted technical debt. *IEEE Transactions on Software Engineering*, 43:1044–1062, 2017.

[13] E. da Silva Maldonado, E. Shihab, and N. Tsantalis. Using natural language processing to automatically detect self-admitted technical debt. *IEEE Transactions on Software Engineering*, 43(11):1044–1062, 2017.

[14] J. Devlin, M. Chang, K. Lee, and K. Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018.

[15] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou. Codebert: A pre-trained model for programming and natural languages, 2020.

[16] R. Gilal, M. Omar, and M. M. Rejab. The key factors contribute to time pressure in software development projects: A review. *International Journal of ADVANCED AND APPLIED SCIENCES*, 10:155–165, 2023.

[17] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.

[18] H. Gu, S. Zhang, Q. Huang, Z. Liao, J. Liu, and D. Lo. Self-admitted technical debts identification: How far are we? In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 804–815. IEEE, 2024.

[19] J. E. Hannay, H. P. Langtangen, C. MacLeod, D. Pfahl, J. Singer, and G. Wilson. How do scientists develop and use scientific software? *2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, pages 1–8, 2009.

[20] P. He, X. Liu, J. Gao, and W. Chen. Deberta: Decoding-enhanced bert with disentangled attention. In *International Conference on Learning Representations*, 2021.

[21] D. Heaton. *Software engineering for enabling scientific software development*. PhD thesis, University of Alabama Libraries, 2015.

[22] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022.

[23] Q. Huang, E. Shihab, X. Xia, D. Lo, and S. Li. Identifying self-admitted technical debt in open source projects using text mining. *Empirical Soft. Engineering*, 2018.

[24] D. Jiang, Y. Liu, S. Liu, J. Zhao, H. Zhang, Z. Gao, X. Zhang, J. Li, and H. Xiong. From clip to dino: Visual encoders shout in multi-modal large language models. *arXiv preprint arXiv:2310.08825*, 2023.

[25] U. Kanewala and J. M. Bieman. Testing scientific software: A systematic literature review. *Information and software technology*, 56(10):1219–1232, 2014.

[26] D. Kelly. Industrial scientific software: a set of interviews on software development. In *Conference of the Centre for Advanced Studies on Collaborative Research*, 2013.

[27] B. Koteska, A. Mishev, and L. Pejov. Quantitative measurement of scientific software quality: Definition of a novel quality model. *International Journal of Software Engineering and Knowledge Engineering*, 28(03):407–425, 2018.

[28] Y. Li, M. Soliman, and P. Avgeriou. Identification and remediation of self-admitted technical debt in issue trackers. In *2020 46th Euromicro conference on software engineering and advanced applications (SEAA)*, pages 495–503. IEEE, 2020.

[29] Y. Li, M. Soliman, and P. Avgeriou. Automatic identification of self-admitted technical debt from four different sources. *Empirical Software Engineering*, 28(3):1–38, 2023.

[30] Y. Li, M. Soliman, P. Avgeriou, and L. Somers. Self-admitted technical debt in the embedded systems industry: An exploratory case study. *IEEE Transactions on Software Engineering*, 49(4):2545–2565, 2022.

[31] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. Roberta: A robustly optimized BERT pretraining approach. *CoRR*, abs/1907.11692, 2019.

[32] I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.

[33] E. d. S. Maldonado and E. Shihab. Detecting and quantifying different types of self-admitted technical debt. In *2015 IEEE 7th International Workshop on Managing Technical Debt (MTD)*, pages 9–15, 2015.

[34] E. L. Melin and N. U. Eisty. Exploring the advances in using machine learning to identify technical debt and self-admitted technical debt. In *2025 IEEE/ACIS 23rd International Conference on Software Engineering Research, Management and Applications (SERA)*, pages 15–22. IEEE, 2025.

[35] G. Miller. A scientist's nightmare: Software problem leads to five retractions. *Science*, 314:1856 – 1857, 2006.

[36] P. Pham, M. Sridharan, M. Esposito, and V. Lenarduzzi. Descriptor: C++ self-admitted technical debt dataset (cppsatd). *IEEE Data Descriptions*, 2025.
[37] G. Pinto, I. Wiese, and L. F. Dias. How do scientists develop scientific software? an external replication. In *2018 IEEE 25th international conference on software analysis, evolution and reengineering (SANER)*, pages 582–591. IEEE, 2018.
[38] A. Potdar and E. Shihab. An exploratory study on self-admitted technical debt. In *International Conference on Software Maintenance and Evolution*. IEEE, 2014.
[39] A. Potdar and E. Shihab. An exploratory study on self-admitted technical debt. In *2014 IEEE International Conference on Software Maintenance and Evolution*, pages 91–100, 2014.
[40] L. Rantala, M. Mäntylä, and V. Lenarduzzi. Keyword-labeled self-admitted technical debt and static code analysis have significant relationship but limited overlap. *Software Quality Journal*, 32(2):391–429, 2024.
[41] L. Rantala, M. Mäntylä, and D. Lo. Prevalence, contents and automatic detection of kl-satd. In *2020 46th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*, pages 385–388. IEEE, 2020.
[42] R. Sharma, R. Shahbazi, F. H. Fard, Z. Codabux, and M. Vidoni. Self-admitted technical debt in r: detection and causes. *Automated Software Engineering*, 29(2):53, 2022.
[43] M. S. Sheikhaei, Y. Tian, S. Wang, and B. Xu. An empirical study on the effectiveness of large language models for satd identification and classification. *Empirical Software Engineering*, 29(6):159, 2024.
[44] E. Sutoyo and A. Capiluppi. Satdaug-a balanced and augmented dataset for detecting self-admitted technical debt. In *2024 IEEE/ACM 21st International Conference on Mining Software Repositories (MSR)*, pages 289–293. IEEE, 2024.
[45] F.-L. Team. The falcon 3 family of open models, Dec. 2024. Accessed: 2025-10-21.
[46] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. Llama 2: Open foundation and fine-tuned chat models, 2023.
[47] D. Tsoukalas, N. Mittas, A. Chatzigeorgiou, D. Kehagias, A. Ampatzoglou, T. Amanatidis, and L. Angelis. Machine learning for technical debt identification. *IEEE Transactions on Software Engineering*, 2021.
[48] L. Xavier, F. Ferreira, R. Brito, and M. T. Valente. Beyond the code: Mining self-admitted technical debt in issue tracker systems. In *Proceedings of the 17th international conference on mining software repositories*, pages 137–146, 2020.
[49] M. Yin, J. Wang, D. Zhu, and C. Gao. A two-stage approach for identifying and interpreting self-admitted technical debt. *Applied Intelligence*, 2023.
[50] F. Zampetti, C. Noiseux, G. Antoniol, F. Khomh, and M. Di Penta. Recommending when design technical debt should be self-admitted. In *2017 IEEE International Conference on Software Maintenance and Evolution (ICSME)*. IEEE, 2017.